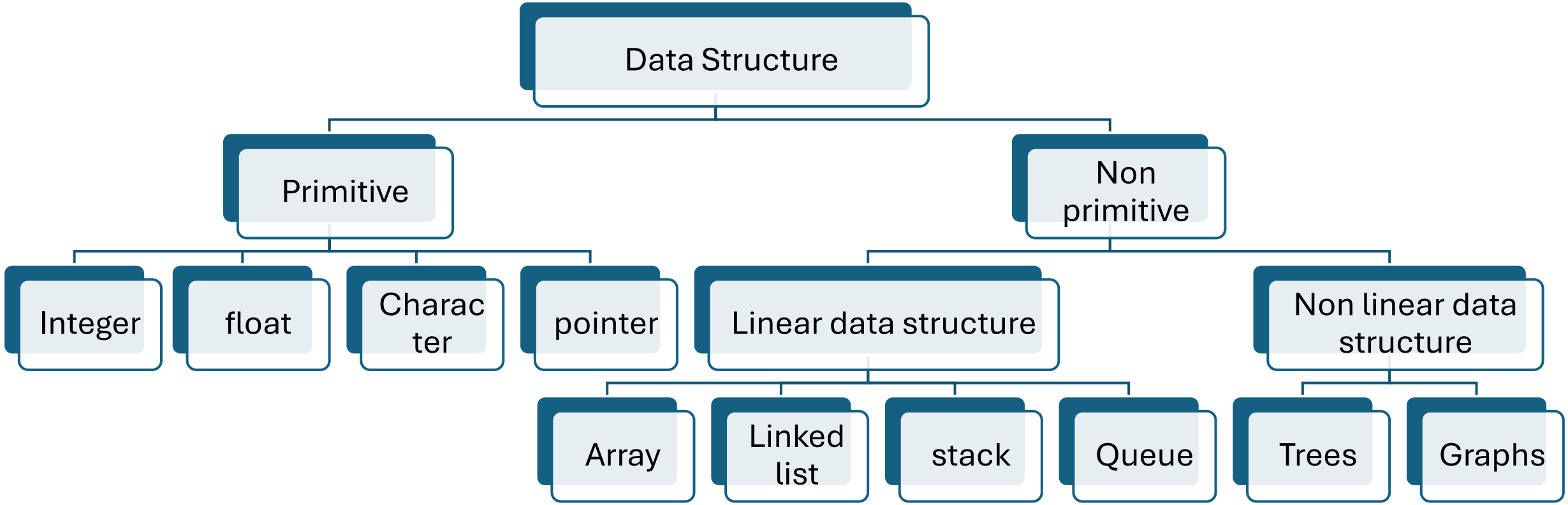
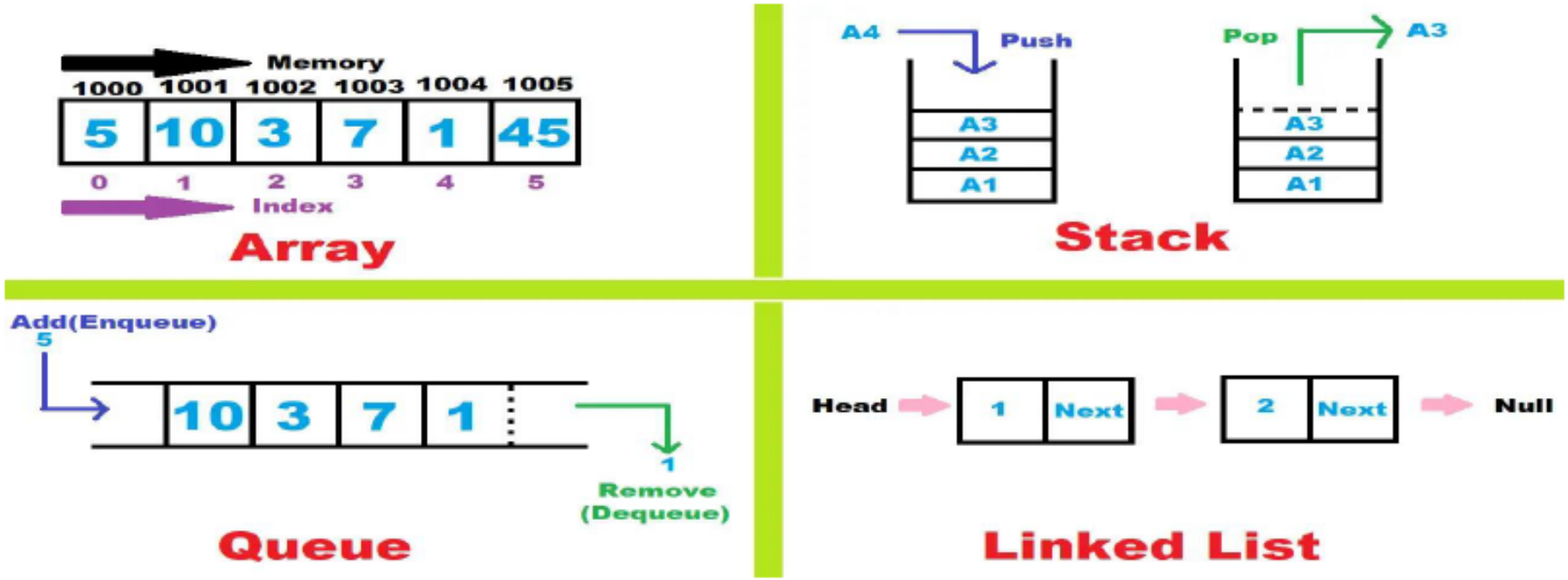
Lecture 4

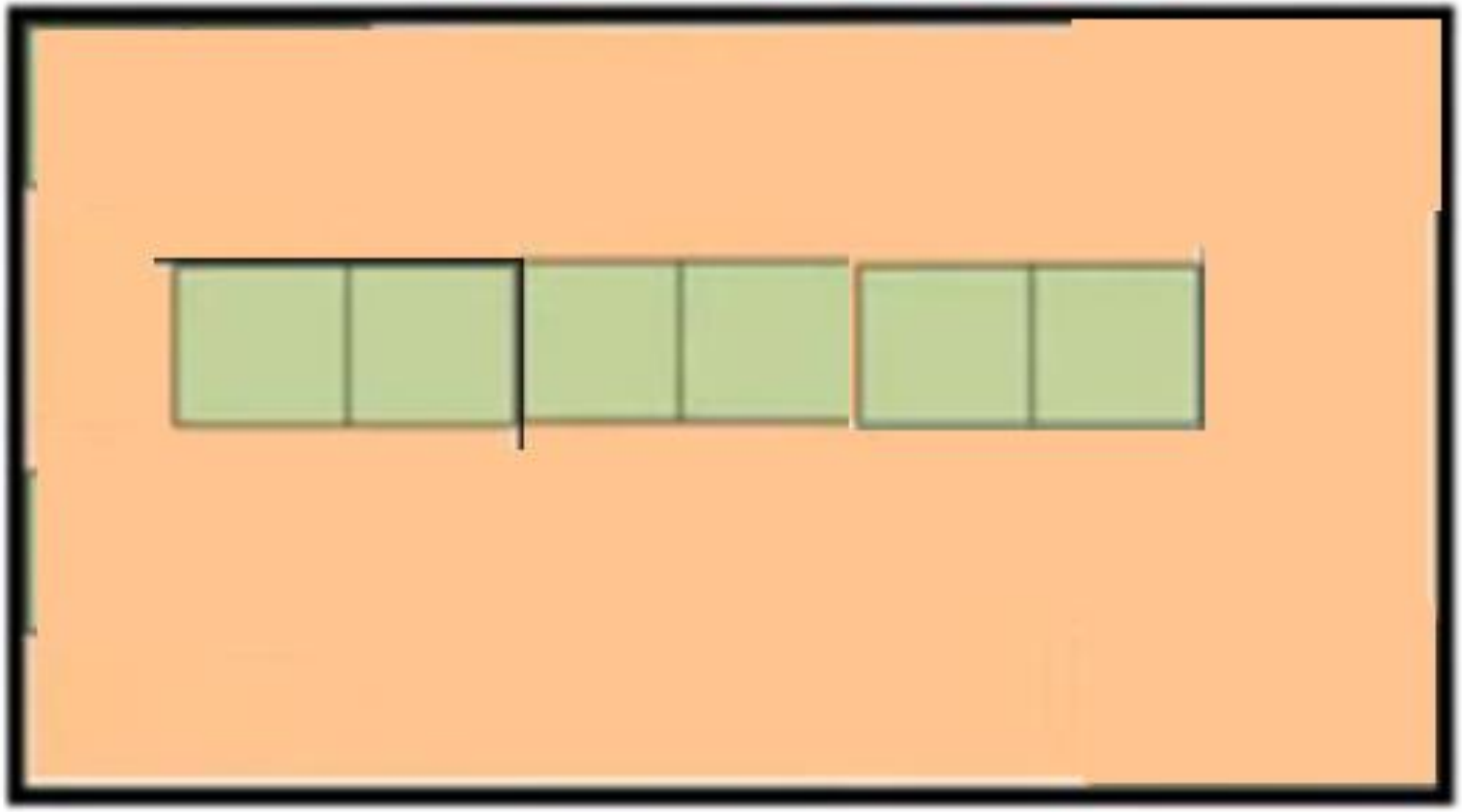

Data Structure
Primitive
Non primitive
Integer
float
Charac ter
pointer
Linear data structure
Non linear data structure
Array
Linked list
stack
Queue
Trees
Graphs

Linear Data Structure

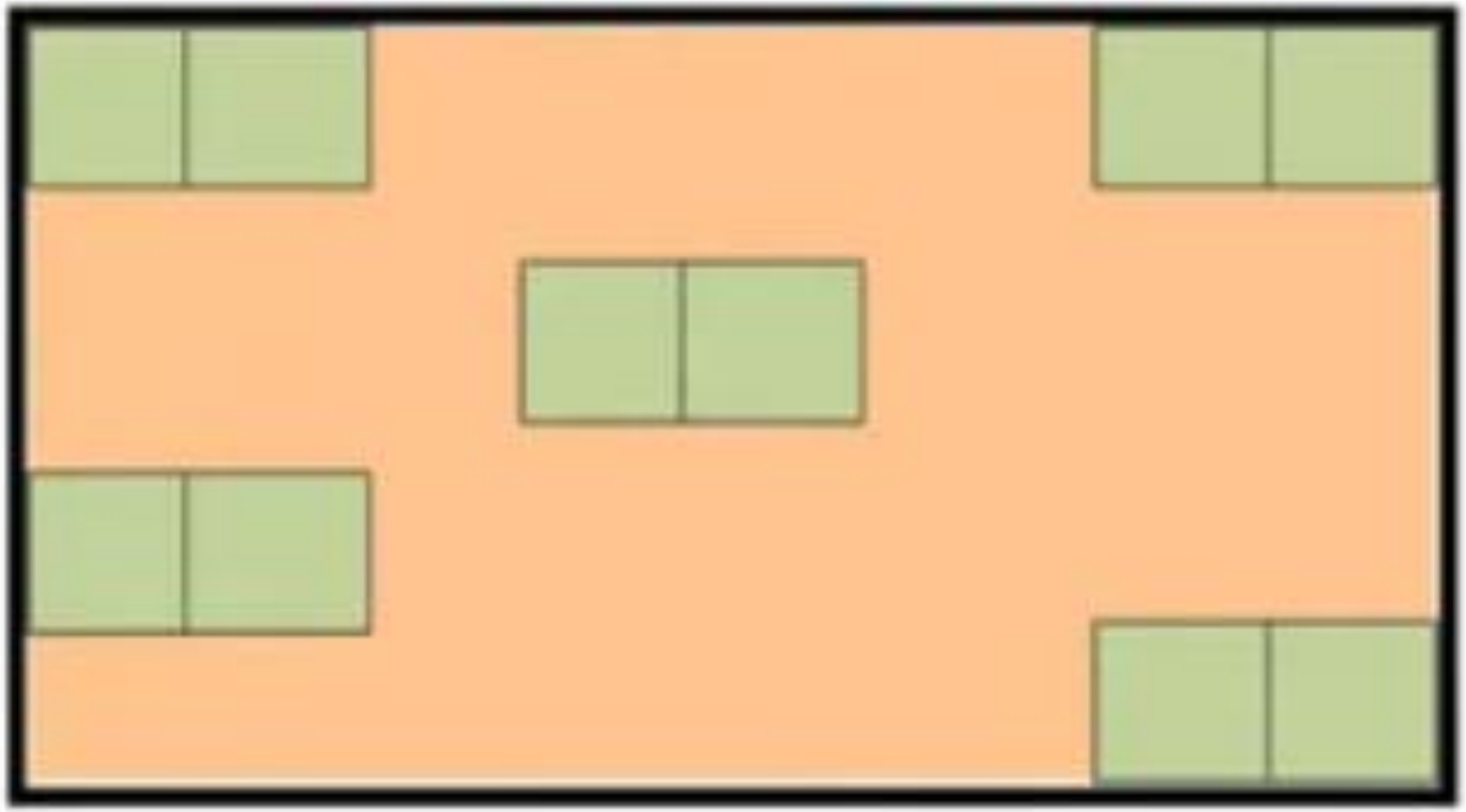
- A data structure is linear when all the elements are arranged in a linear (sequential) order.
- Each element is having only one predecessor and one successor. There is one exception the first element has no predecessor and last element has no successor.



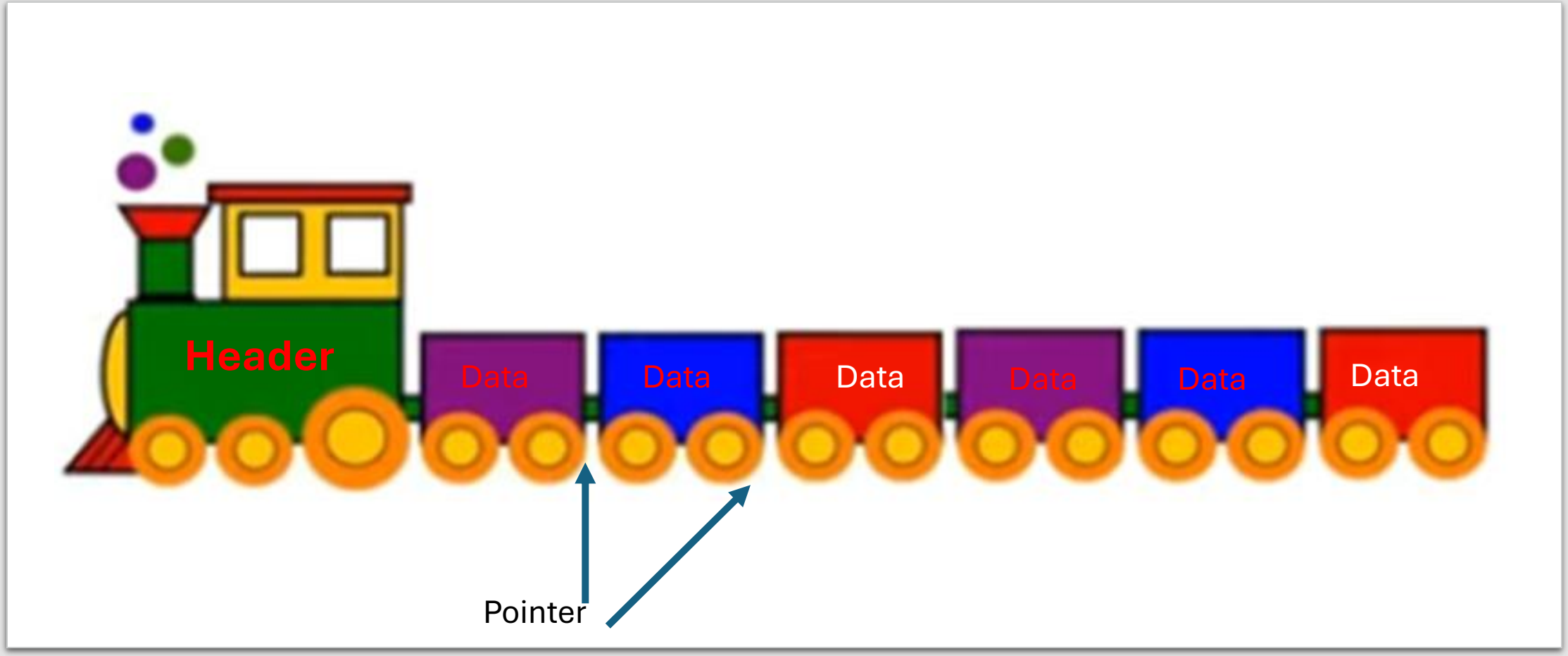
MEMORY



MEMORY

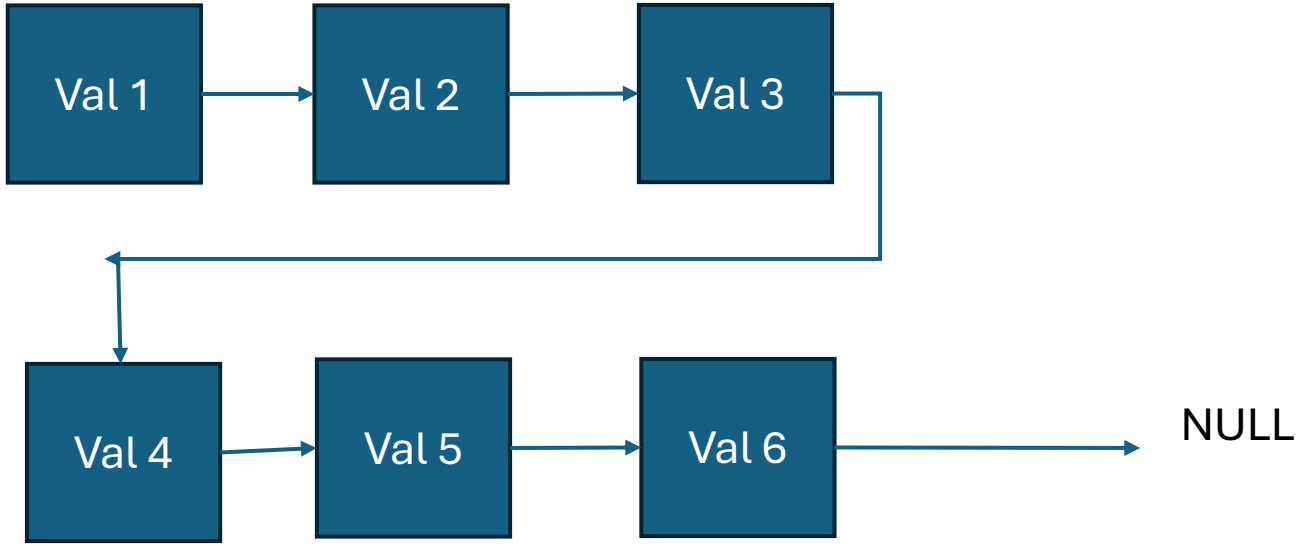


Linked list

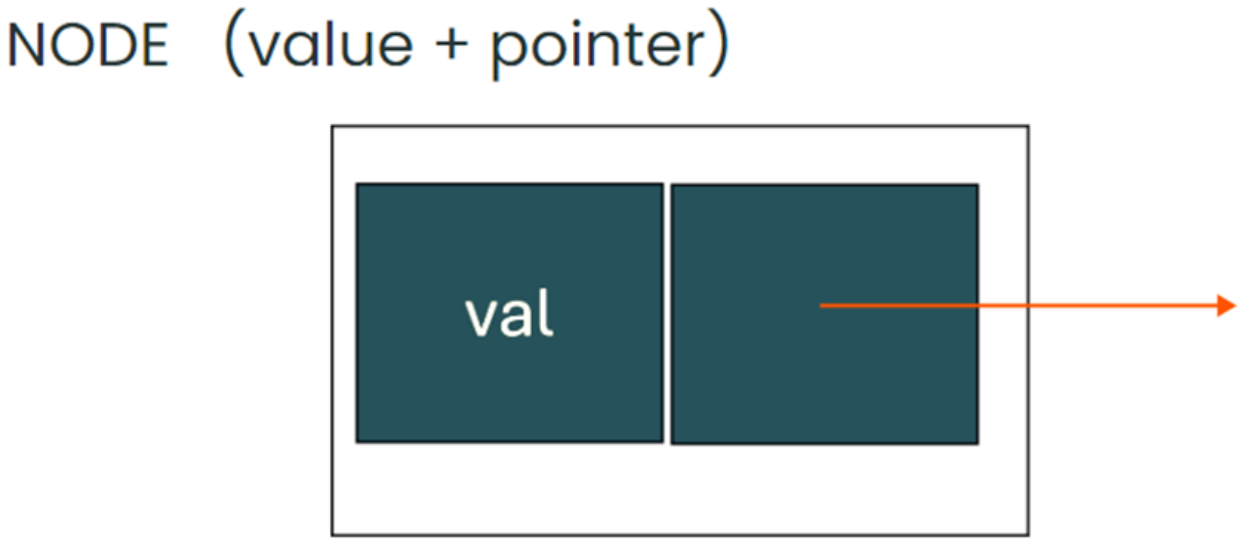


Linked List

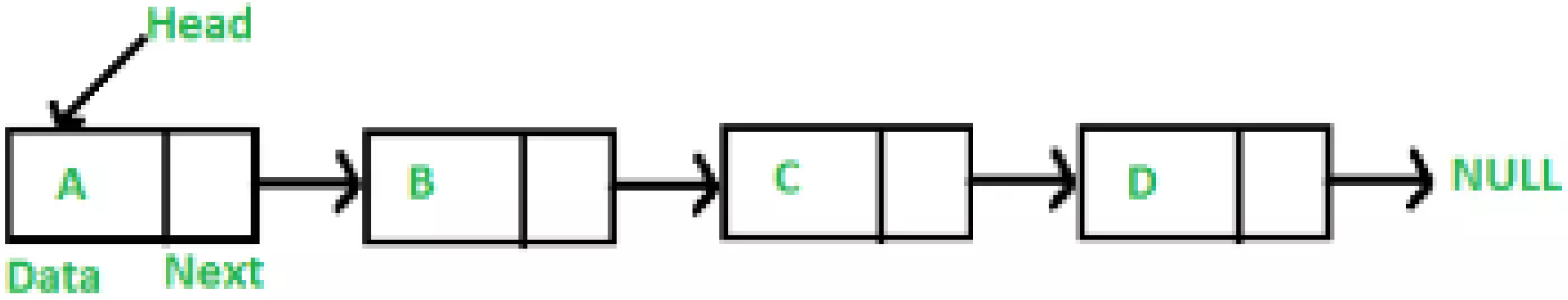
- That is as an array but doesn’t need to allocate a specific size in the first, that depend on adding values in elements and each element has a pointer to the next element.



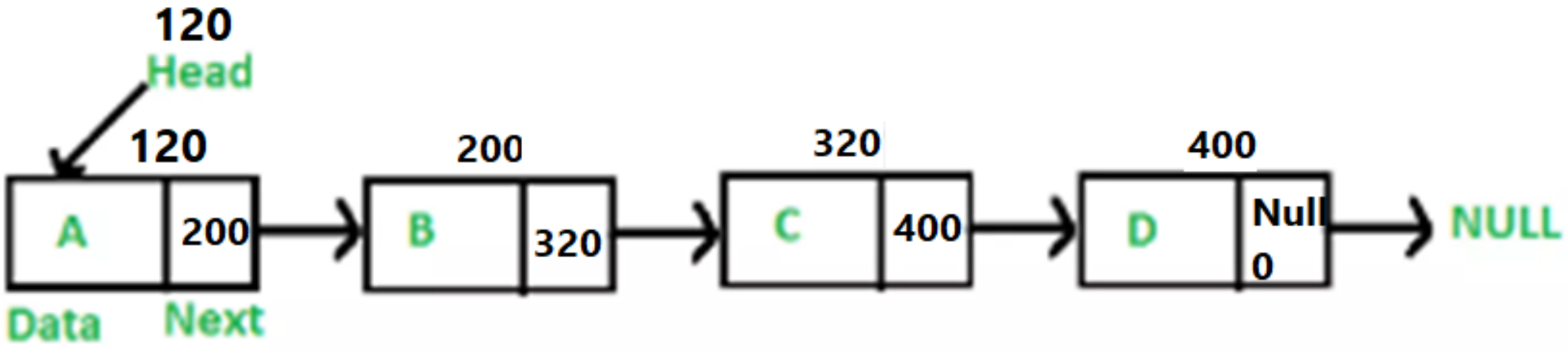
- The structure of a linked list node. ***Each node*** of a linked list contains a ***single data element*** and a ***pointer to the next node*** in the list.

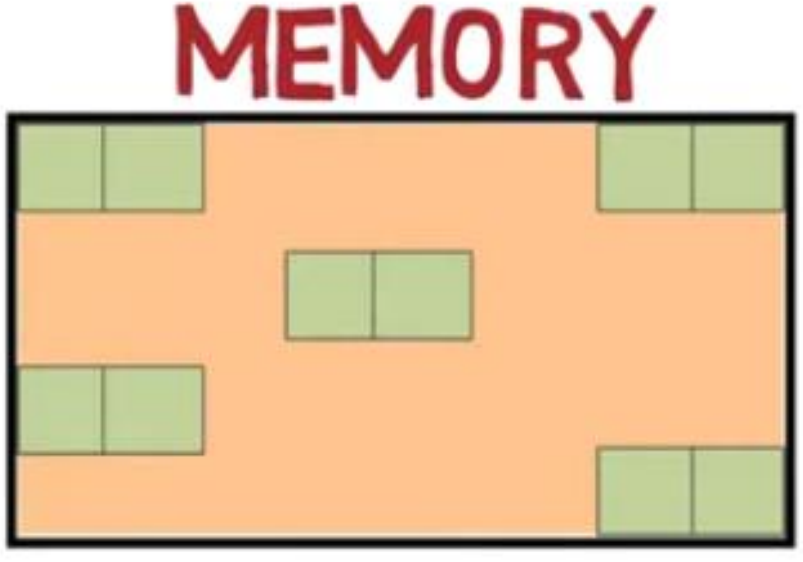


- The structure of a linked list node. ***Each node*** of a linked list contains a ***single data element*** and a ***pointer to the next node*** in the list and there is a head to point to the first node (head contain the location of first node).



Example:


120
Head
120
200
320
400
A
200
B
320
C
400
D
Null
0
NULL
Data
Next


MEMORY

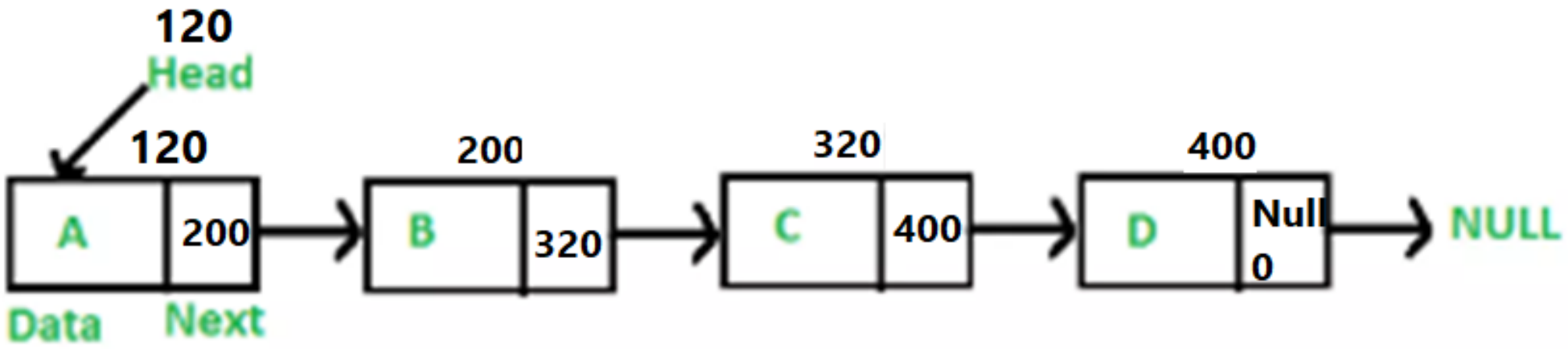
Types of linked list

Types of linked list

- Singly linked lists.
- Doubly linked lists.
- Circular linked lists.
- Circular doubly linked lists

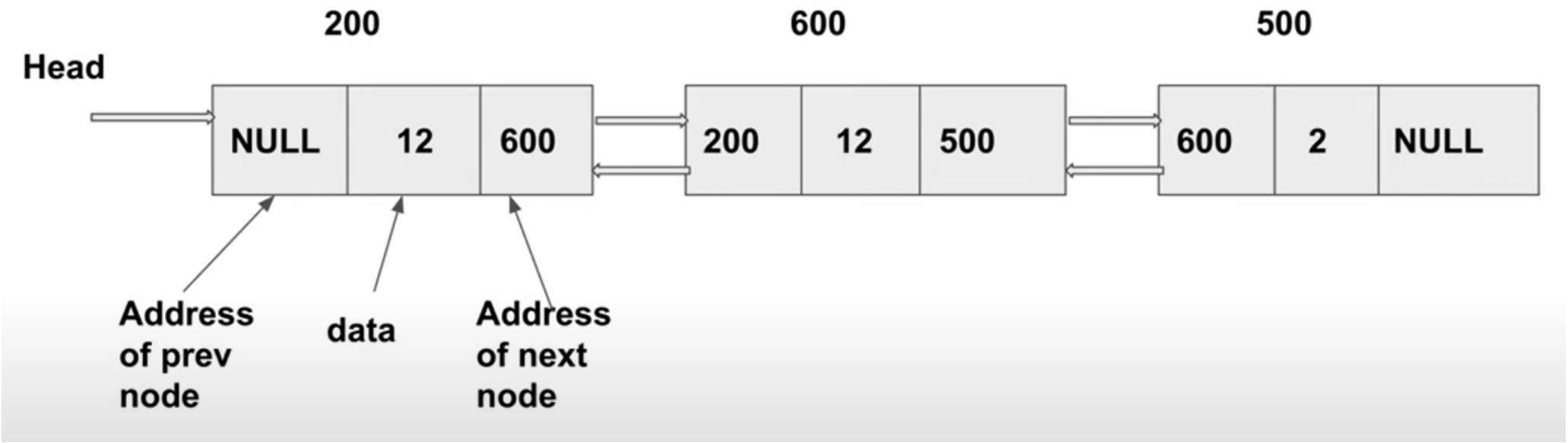
Single linked list

A singly-linked list is the simplest type of linked list, where each node contains some data and a reference to the next node in the sequence. They can only be traversed in a single direction - from the head (the first node) to the tail (the last node).



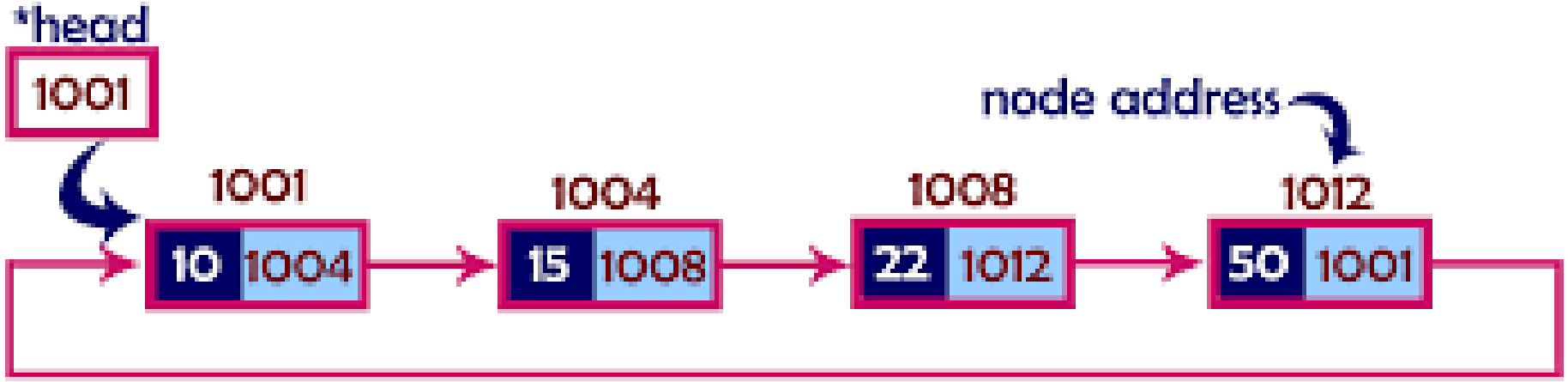
DOUBLY LINKED LIST

A doubly linked list is a more complex data structure than a singly linked list, but it offers several advantages. The main advantage of a doubly linked list is that it allows for efficient traversal of the list in both directions. This is because each node in the list contains a pointer to the previous node and a pointer to the next node. This allows for quick and easy insertion and deletion of nodes from the list, as well as efficient traversal of the list in both directions.

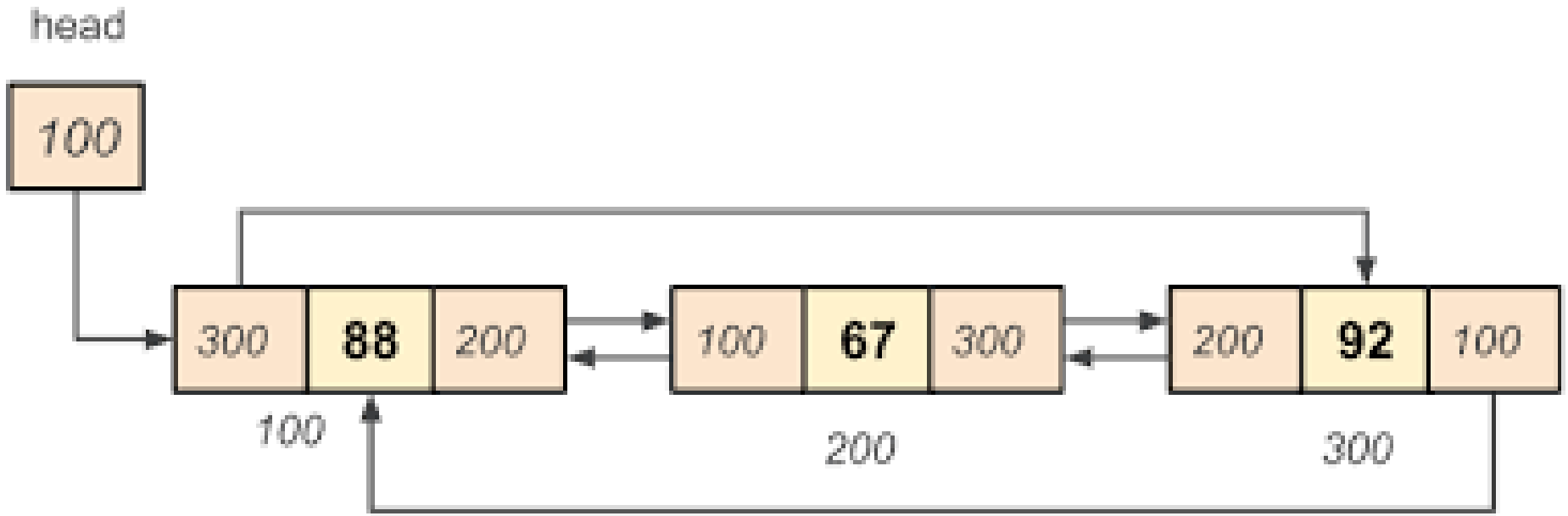


Circular linked list

A circular linked list is a special type of linked list where all the nodes are connected to form a circle. Unlike a regular linked list, which ends with a node pointing to NULL, the last node in a circular linked list points back to the first node. This means that you can keep traversing the list without ever reaching a NULL value.



Double circular linked list



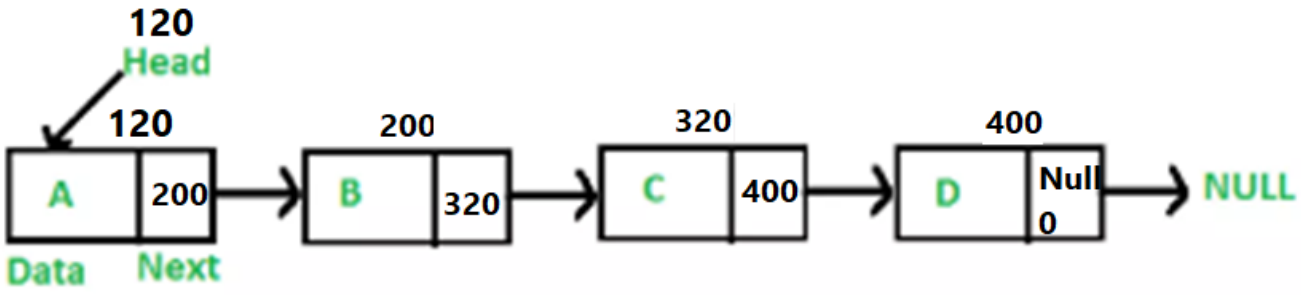
Operation of single linked list

1. Traversal of Singly Linked List

Traversal in a linked list means visiting each node and performing operations like printing or processing data.

Step-by-step approach:

1. **Initialize a pointer** (current) to the head of the list.
2. **Loop through the list** using a while loop until current becomes NULL.
3. **Process each node** (e.g., print its data).
4. **Move to the next node** by updating current = current->next.

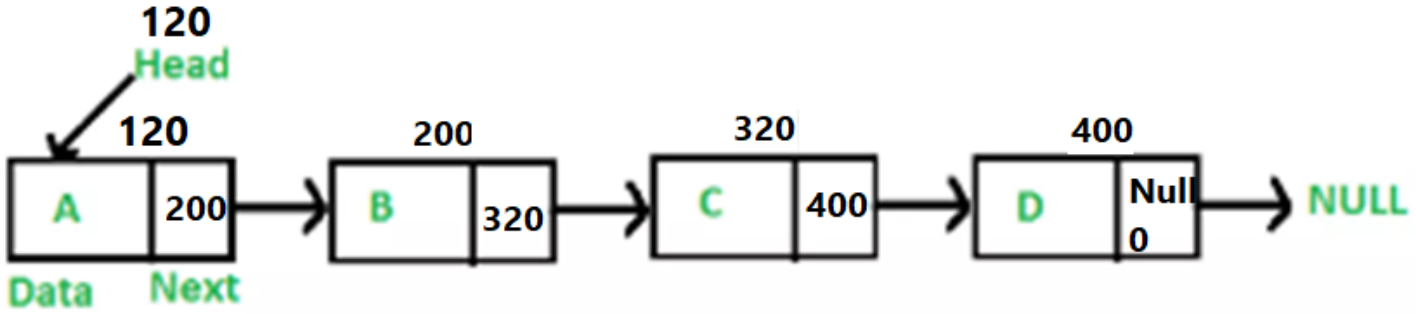


2. Searching in Singly Linked List

Searching in a Singly Linked List refers to the process of looking for a specific element or value within the elements of the linked list.

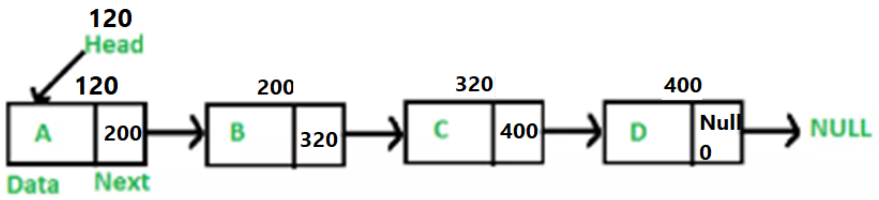
Step-by-step approach:

1. **Start from the head** of the linked list.
2. **Check each node's data**:
   - If it matches the target value, **r**eturn true (element found).
   - Otherwise, move to the next node.
3. **Repeat until** the end (NULL) is reached.
4. If no match is found, **return false**.



3. Length of Singly Linked List

Finding the length of a **Singly Linked List** means counting the total number of nodes.



Step-by-step approach:

1. **Initialize** a counter (length = 0).
2. **Start from the head**, assign it to current.
3. **Traverse the list**:
   - **Increment length** for each node.
   - **Move to the next node** (current = current->next).
4. **Return the final length** when current becomes NULL.

4. Insertion in Singly Linked List

- Insertion is a fundamental operation in linked lists that involves adding a new node to the list. There are several scenarios for insertion:

- **a. Insertion at the Beginning of Singly Linked List:** Insertion at the beginning involves adding a new node before the current head, making it the new head.

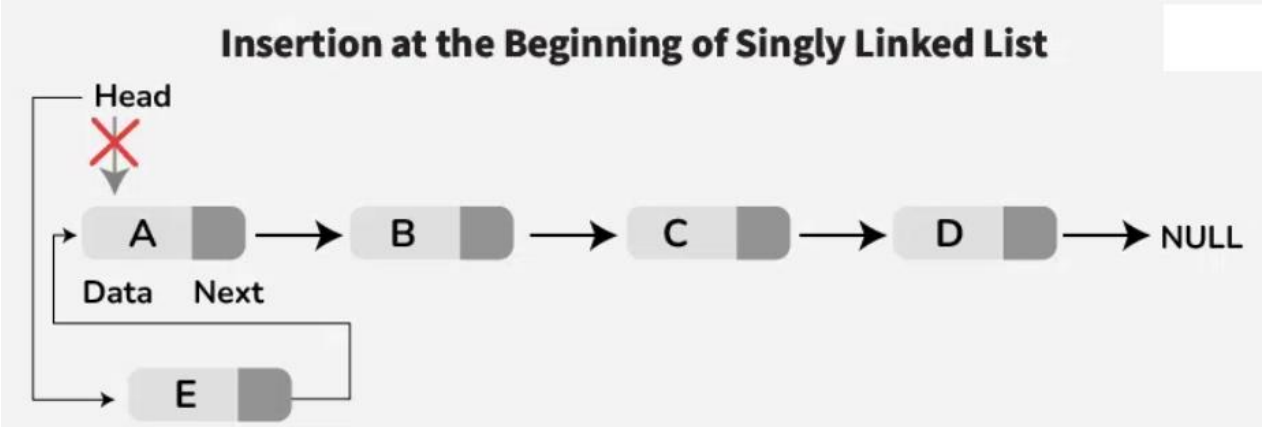
Step-by-step approach:

•Create a new node with the given value.

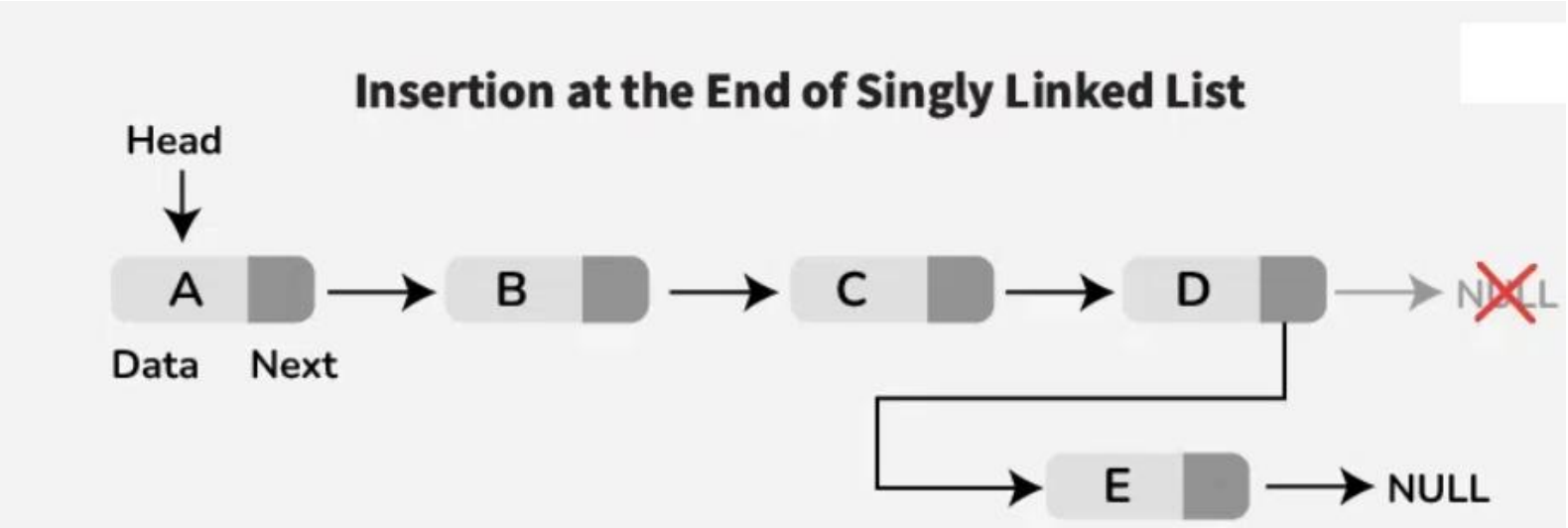
•Set the **next** pointer of the new node to the **current head**.

•Move the head to point to the new node.

•Return the new head of the linked list.



- **b. Insertion at the End of Singly Linked List:** To insert a node at the end of the list, traverse the list until the last node is reached, and then link the new node to the current last node
- **Step-by-step approach:**
- Create a new node with the given value.
- Check if the list is empty:
  - If it is, make the new node the head and return.
- Traverse the list until the last node is reached.
- Link the new node to the current last node by setting the last node's next pointer to the new node.



- **c. Insertion at a Specific Position of the Singly Linked List:** To insert a node at a specific position, traverse the list to the desired position, link the new node to the next node, and update the links accordingly.

Step-by-step approach:

•**Create a new node** and assign it a value.

•**If inserting at the beginning** (position = 1):

  •Point the new node's next to the current head.

  •Update the head to the new node.

  •**Return** (Insertion done).

•**Otherwise, traverse the list**:

  •Start from the head and move to the (position - 1)th node (just before the desired position).

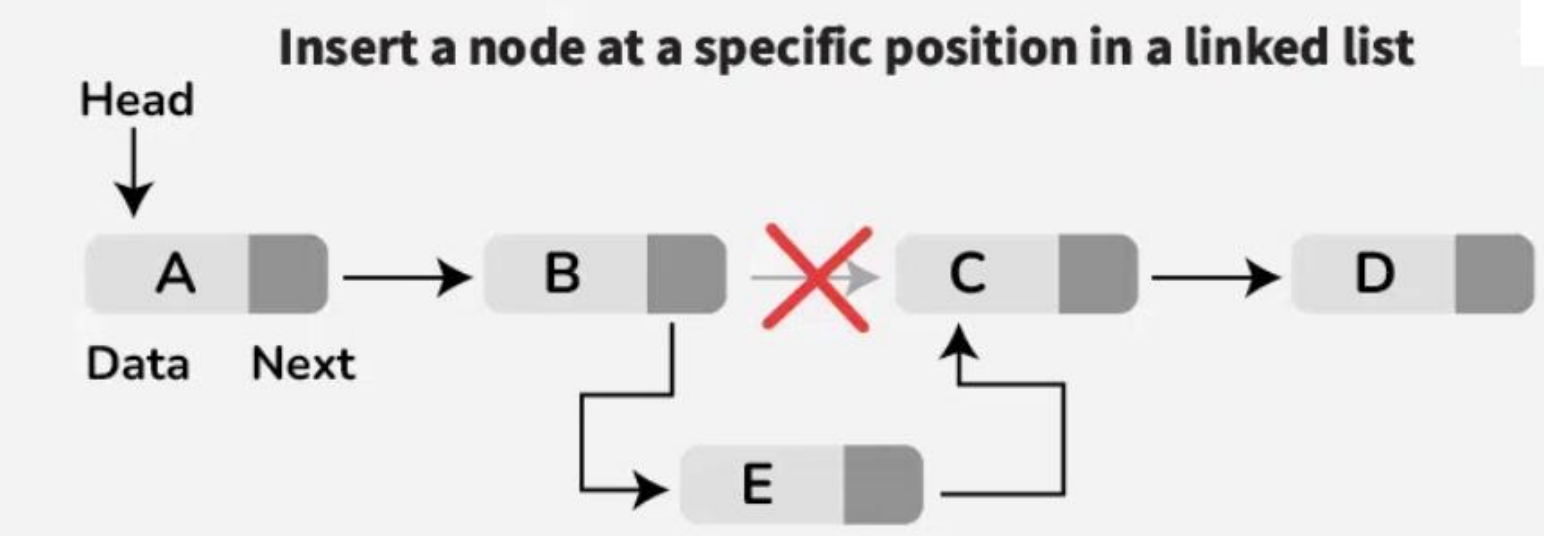
  •If the position is beyond the list length, **return an error or append at the end**.

•**Insert the new node**:

  •Point the new node's next to the next node of the current position.

  •Update the previous node's next to the new node.

•**Return the updated list.**



- **5. Deletion in Singly Linked List**

- Deletion involves removing a node from the linked list. Similar to insertion, there are different scenarios for deletion:

- **a. Deletion at the Beginning of Singly Linked List:** To delete the first node, update the head to point to the second node in the list.

- **Steps-by-step approach:**

- Check if the head is **NULL**.

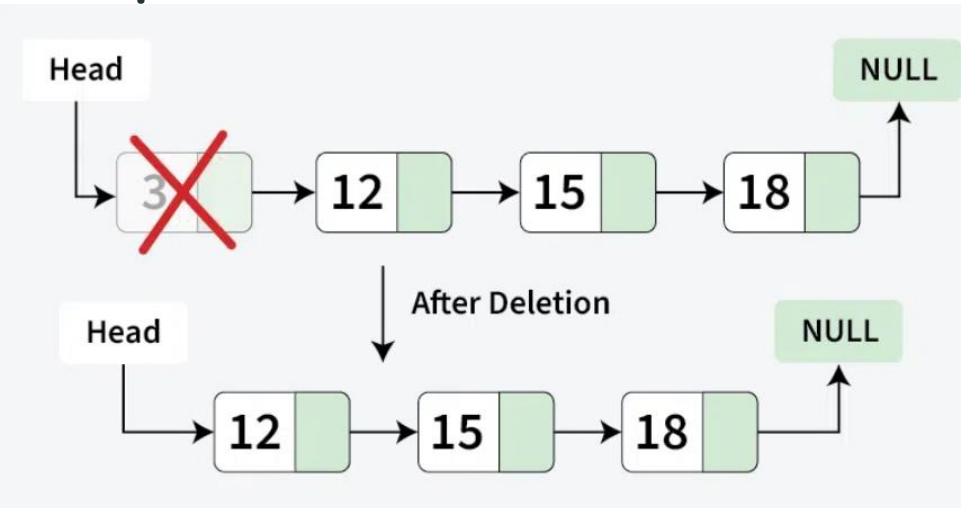
  - If it is, return **NULL** (the list is empty).

- Store the current head node in a temporary variable **temp**.

- Move the head pointer to the next node.

- Delete the temporary node.

- Return the new head of the linked list.



Deletion at beginning in a Linked List

- **b. Deletion at the End of Singly Linked List:** To delete the last node, traverse the list until the second-to-last node and update its next field to None.

Step-by-step approach:

•Check if the head is **NULL**.

- If it is, return NULL (the list is empty).

•Check if the head's **next** is **NULL** (only one node in the list).

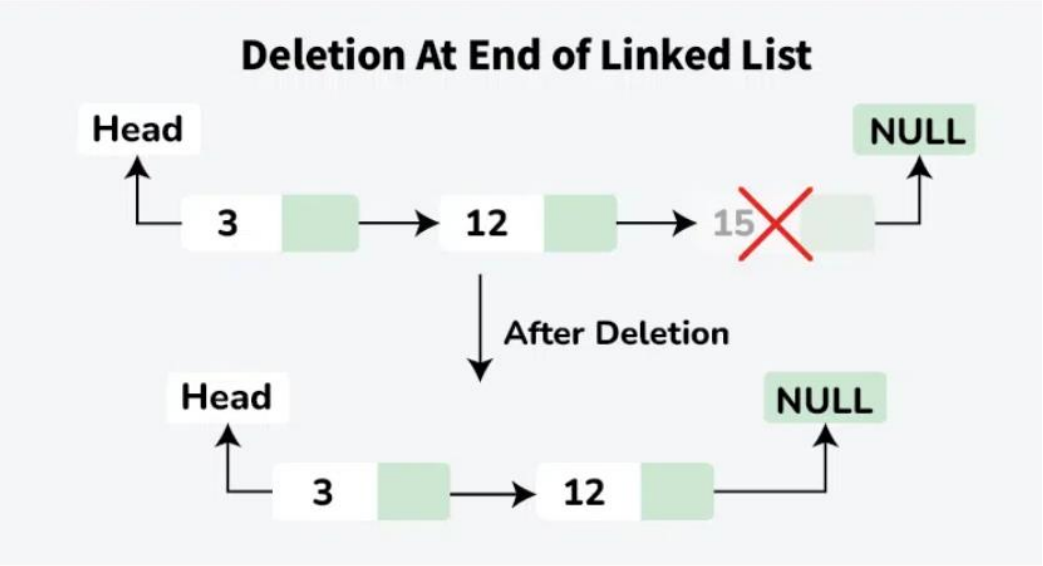
- If true, delete the head and return **NULL**.

•Traverse the list to find the second last node (**second_last**).

•Delete the last node (the node after **second_last**).

•Set the **next** pointer of the second last node to **NULL**.

•Return the head of the linked list.



Deletion at the end of linked list

- **c. Deletion at a Specific Position of Singly Linked List:** To delete a node at a specific position, traverse the list to the desired position, update the links to bypass the node to be deleted.

Step-by-step approach:

•Check if the list is empty or the position is invalid, return if so.

•If the head needs to be deleted, update the head and delete the node.

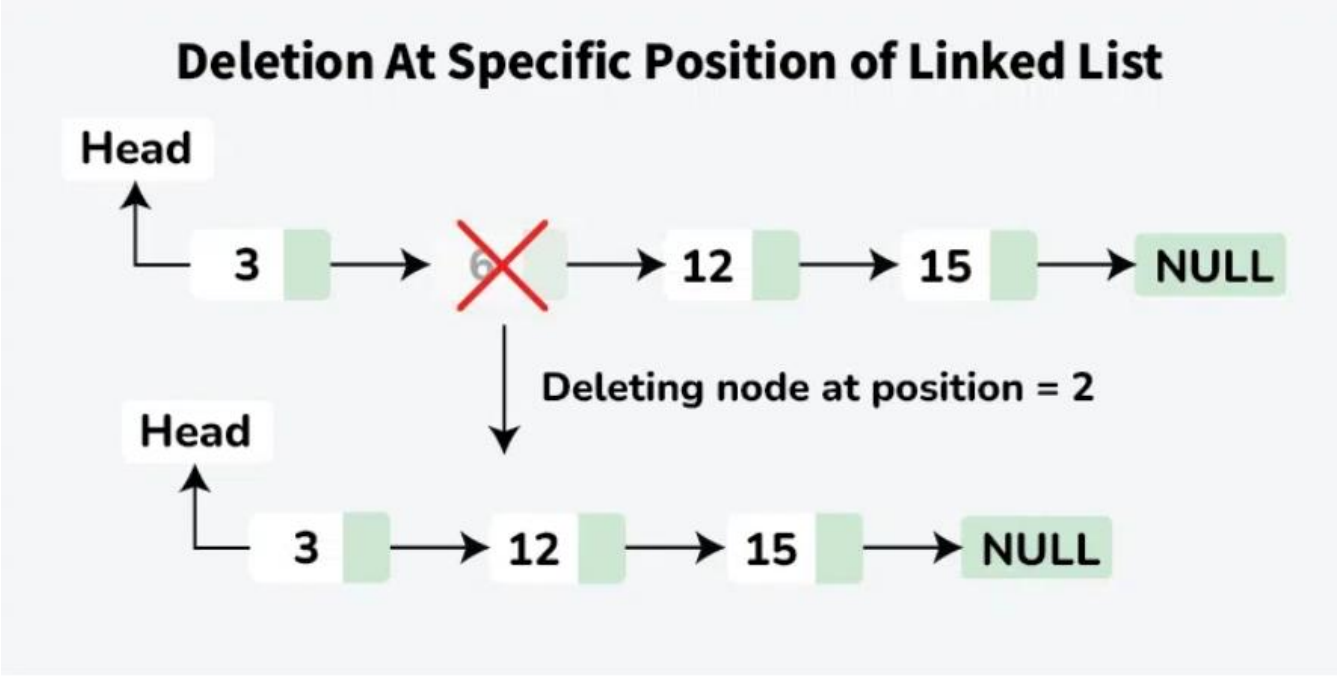
•Traverse to the node before the position to be deleted.

•If the position is out of range, return.

•Store the node to be deleted.

•Update the links to bypass the node.

•Delete the stored node.



Delete a Linked List node at a given position

6. Modify a Singly Linked List

Updating in a Singly Linked List means modifying the value of a node at a given position.

Step-by-step approach:

- **Start from the head** of the list.
- **Traverse to the required position** (move current node to position).
- **Check if the position is valid**:
  - If the position is out of bounds, **return an error**.
- **Update the node's data** with the new value.
- **Return the modified list.**

DOUBLY LINKED LIST

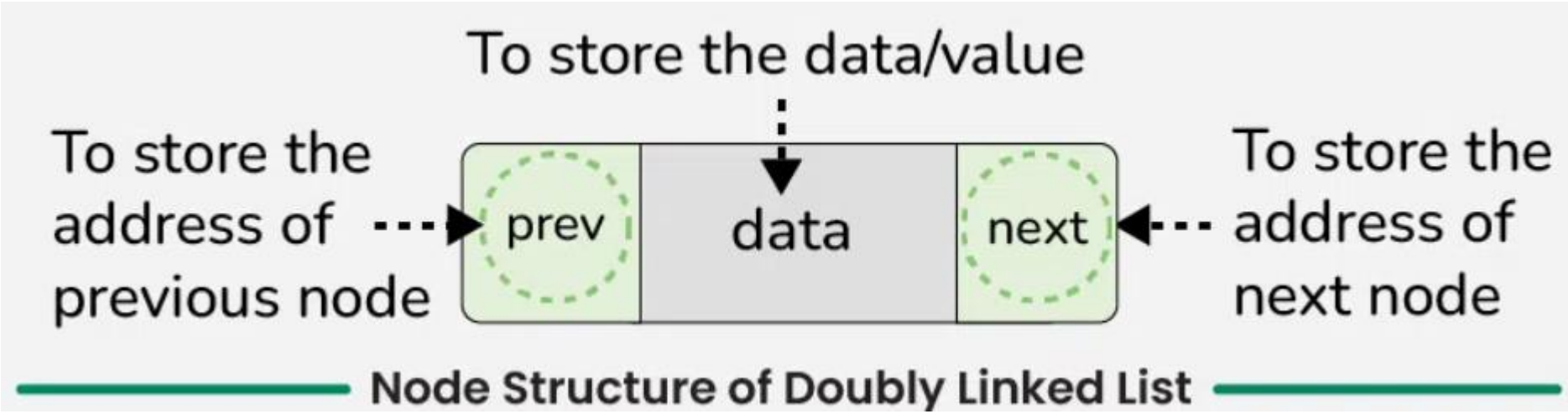
DOUBLY LINKED LIST

- In a data structure, a doubly linked list is represented using nodes that have three fields:

1. Data

2. A pointer to the next node (**next**)

3. A pointer to the previous node (**prev**)



Double linked list functions

Double linked list functions

- Traversal in Doubly Linked List
- Finding Length of Doubly Linked List
- Insertion in a Doubly Linked List
- Deletion in a Doubly Linked List

Traversal in Doubly Linked List

Traversal in a **Doubly Linked List** involves visiting each node, using the forward (next) and backward (prev) pointers.

Step-by-Step Approach for Traversal:

1. **Start from the head** of the list.
2. **Traverse forward**:
   - Visit the current node and process its data (e.g., print it).
   - Move to the next node using current = current->next.
   - Repeat the process until the end of the list (current == NULL).
3. **Optionally, traverse backward**:
   - Start from the tail (last node).
   - Visit the current node and process its data.
   - Move to the previous node using current = current->prev.
   - Repeat the process until the beginning of the list (current == NULL).

Traversal is useful for displaying

Finding Length of Doubly Linked List

A Doubly Linked List (DLL) is a type of linked list where each node has two pointers:

1. One pointing to the next node in the sequence.
2. One pointing to the previous node in the sequence.

To find the length of a doubly linked list, we need to traverse the list while counting the nodes.

Step-by-Step Approach for finding length:

1. **Initialize a counter**: Start with a counter variable (count = 0).
2. **Set a pointer to the head node**: Use a pointer (current) and initialize it to the head of the linked list.
3. **Traverse the list**:
   - While the pointer (current) is not NULL, increment the count by 1.
   - Move to the next node (current = current.next).
4. **Stop at the end of the list**: When the pointer reaches NULL, stop the loop.
5. **Return the count**: The final value of count gives the length of the doubly linked list.

Insertion in a Doubly Linked List

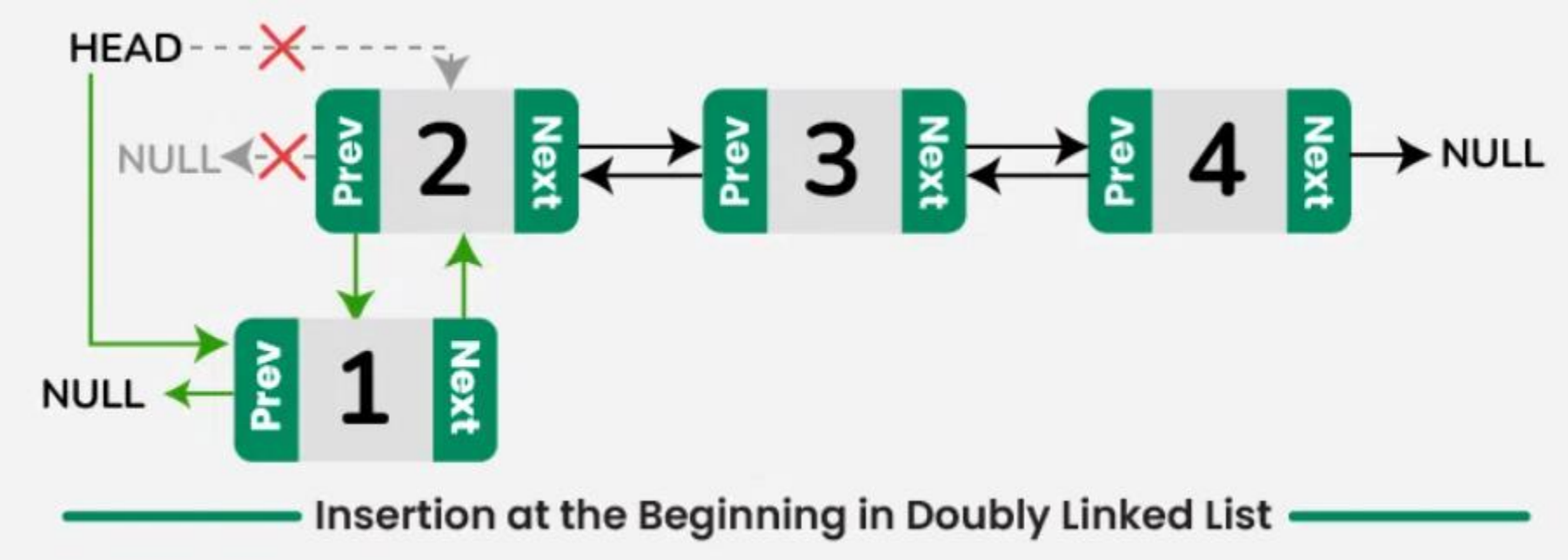
Insertion in a **Doubly Linked List (DLL)** involves adding a new node at a specific position while maintaining the connections between nodes. Since each node contains a pointer to both the previous and next node, insertion requires adjusting these pointers carefully.

There are three primary types of insertion in a DLL:

1. **Insertion at the Beginning**
2. **Insertion at the End**
3. **Insertion at a Specific Position**

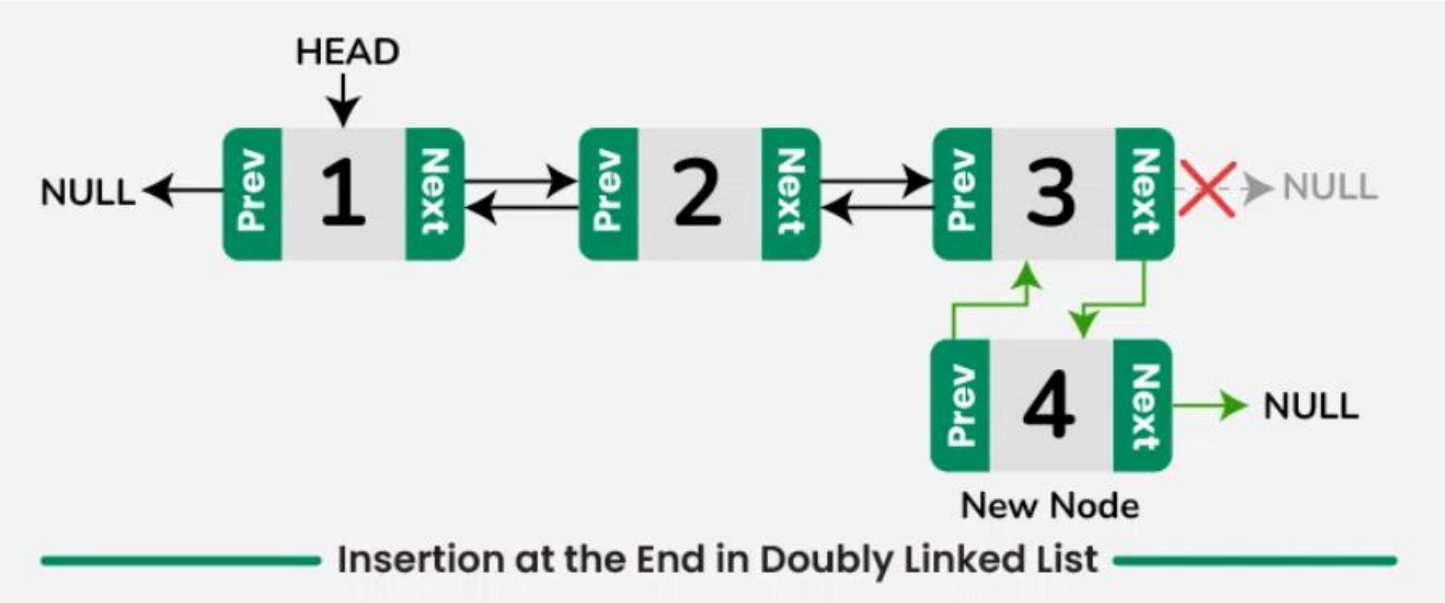
1. Insertion at the Beginning

1.Create a new node with the given data.
2.Set the next pointer of the new node to the current head.
3.If the list is not empty, update the prev pointer of the current head to point to the new node.
4.Update the head of the list to the new node.



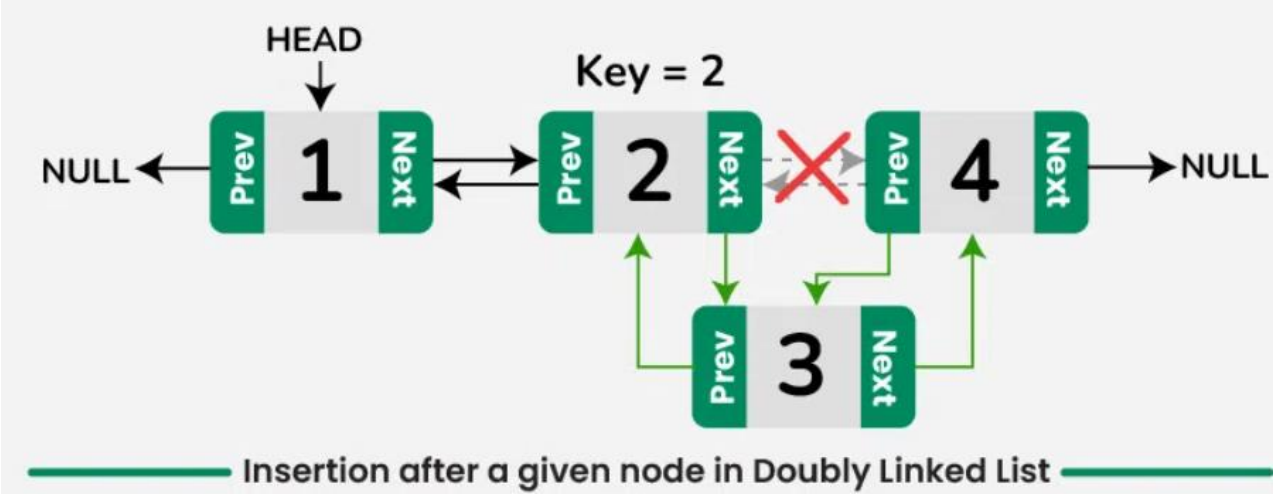
2. Insertion at the End

1.Create a new node with the given data.
2.If the list is empty, set the new node as the head.
3.Traverse the list until the last node is found.
4.Set the next pointer of the last node to the new node.
5.Set the previous pointer of the new node to the last node.



3. Insertion at a Specific Position

1.Create a new node with the given data.
2.If inserting at the beginning, follow the steps for insertion at the start.
3.Traverse the list to find the node after which insertion is needed.
4.Set the next pointer of the new node to the next node of the current position.
5.Set the prev pointer of the new node to the current node.
6.Update the prev pointer of the next node to point to the new node (if it exists).
7.Update the next pointer of the previous node to point to the new node.



Deletion in a Doubly Linked List

- Deletion in a **Doubly Linked List (DLL)** involves removing a node while maintaining the integrity of the list. Since each node contains pointers to both its previous and next nodes, deletion requires careful pointer adjustments to ensure no broken links occur.
- **Types of Deletion in a Doubly Linked List**
- **1. Deletion at the Beginning**
- **2. Deletion at the End**
- **3. Deletion at a Specific Position**

1. Deletion at the Beginning

1. Check if the list is empty; if it is, return as there is nothing to delete.
2. Store the current head node in a temporary variable.
3. Move the head pointer to the next node.
4. If the new head exists, update its prev pointer to NULL.
5. Delete the old head node to free memory.

2. Deletion at the End

1. Check if the list is empty; if it is, return.
2. Traverse the list to find the last node.
3. Store the last node in a temporary variable.
4. Update the next pointer of the second-last node to NULL, making it the new tail.
5. Delete the last node to free memory.

3. Deletion at a Specific Position

1. Check if the list is empty; if it is, return.
2. Traverse the list to find the node to be deleted.
3. Store the node to be deleted in a temporary variable.
4. Update the next pointer of the previous node to point to the next node.
5. Update the prev pointer of the next node to point to the previous node (if it exists).
6. Delete the target node to free memory.